

Computer Science & IT

Database Management System



**File organization
And
Indexing**

Lecture No. 04



By- Vishal Sir

Recap of Previous Lecture



✓ Topic

Types of index

✓ Topic

Primary index

✓ Topic

Clustering index

✓ Topic

Secondary index

✓ Topic

Multi level index

Topics to be Covered



- ✓ **Topic** Dynamic Multi level index
- ✓ **Topic** Structure of B tree
- ✓ **Topic** Structure of B+ tree





Topic : Types of Index

Single level index

- Primary index ✓
- Clustering index ✓
- Secondary index ✓

Multi-level Index }

Dynamic Multi-level index

- B tree
- B+ tree



Topic : Primary Index

If ① Search key attribute is a key {ie. Unique values}
and ② File is ordered based on that search key attribute,
then Index file created on such search key
is called primary index.



Topic : Primary Index

Search key (Eid) Pointer

1	
9	
13	
20	
27	
-	

B₁

B₂

Search key

Unique
E.M.P.

Eid	E.M.P.
1	
7	
9	
10	
13	
16	
20	
24	
27	
30	

B₁

B₂

B₃

B₄

B₅

Eid is a key attribute in EMP table.

And

Records of the EMP table are ordered based on Eid

Index is a Primary index.

Index file
(Primary index)

{ It may be Sparse or dense }

{ In general, Primary index is Sparse }



Topic : Clustering Index

If ① Search key attribute is non-key attribute { i.e. not unique values }
and ② File is ordered on search key attribute,
then index file created on such search key is called
a Clustering index



Topic : Clustering Index

Search Key

not unique EMP

Search Key
Dept-id
Pointer

1	
2	
4	
5	
7	

Dept-id	Eid	- - - -
1	7	
2	13	
2	1	
2	5	
4	4	
5	9	
5	15	
7	6	
7	20	
7	12	

B₁
B₂
B₃
B₄
B₅

Block Anchor

Points to the 1st record corresponding to the search key

⊕ Dept-id is a non-key attribute in EMP table

4 Records of the EMP table are physically ordered based on Dept-id

∞ Index is Clustering index

Index file (Clustering)

{ Clustering index will always be Sparse index }



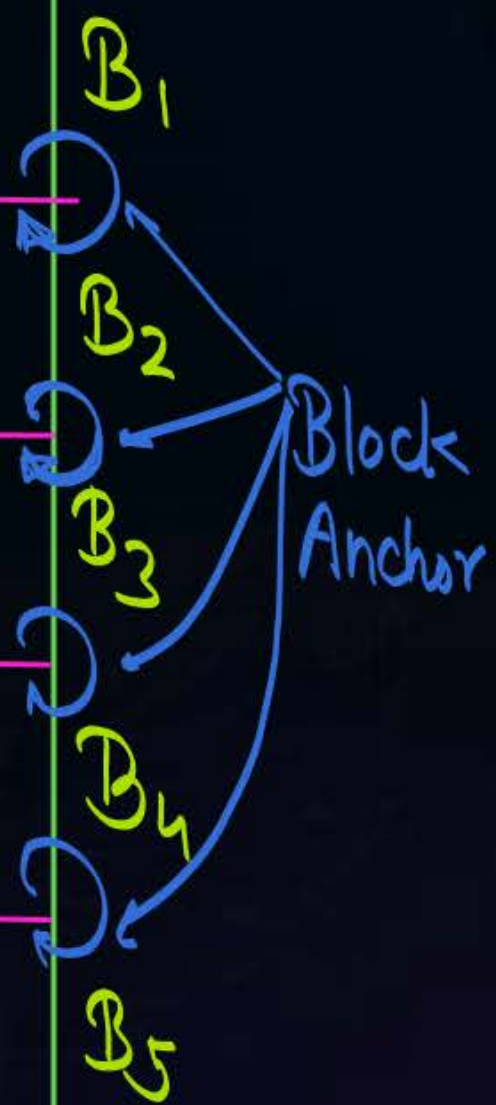
Topic : Clustering Index

EMP

Search key
Dept-id
Pointer

1	
2	

Dept-id	Eid	- - - -
1	7	
2	13	
2	1	
2	5	
4 ^x	4	
5	9	
5	15	
7	6	
7	20	
7	12	



⊕ Dept-id is a non-key attribute in EMP table

Records of the EMP table are physically ordered based on Dept-id

∴ Index is Clustering index

This is add^r of the first record with search key = 2
If the record is the last record of the block, then we don't know whether it is the last record w.r.t. search key = 2 or not.

So we will use block anchor
And we will access the next block of the file to check if there are more records w.r.t. search key = 2 or not

Note :- $\rightarrow$ ① Primary index can be sparse index or dense index { generally, Primary index is sparse }

② Clustering index is always a Sparse index.

③ At most one primary index is possible on a database file { because file must be ordered based on search key }

④ At most one Clustering index is possible on a database file. { because file must be ordered based on search key }

Note ⑤ On a database file we can either have a Primary index or we can have a Clustering index but not both.
Because at a time file can be ordered w.r.t. at most one attribute



Topic : Secondary Index

Search key
Values may or
may not be unique
in database file

- If file is unordered based on search key, then index file created on such search key is called secondary index



Topic : Secondary Index

file is unordered
based on Pass-No.
∴ Secondary index

Pass-No	Pointer
1254	
6102	

file is unordered
based on rating
∴ Secondary index

Rating	Pointer

Pass-No.	Flyer database	Rating
4		1
1		2
9		3
2		2
13		5
7		3
10		5
12		6



Topic : Secondary Index

file is unordered
based on Pass-No.
∴ Secondary index

Pass-No.	Pointer
1254	
67019	

file is unordered
based on rating
∴ Secondary index

Rating	Pointer
1254	
67019	

Pass-No.	Flyer database	Rating
4		1
1		2
9		3
2		2
13		5
7		3
10		5
12		5

Note:- When file is unordered based on search key. then our first objective will be to provide an order using index file so that binary search can be applied.

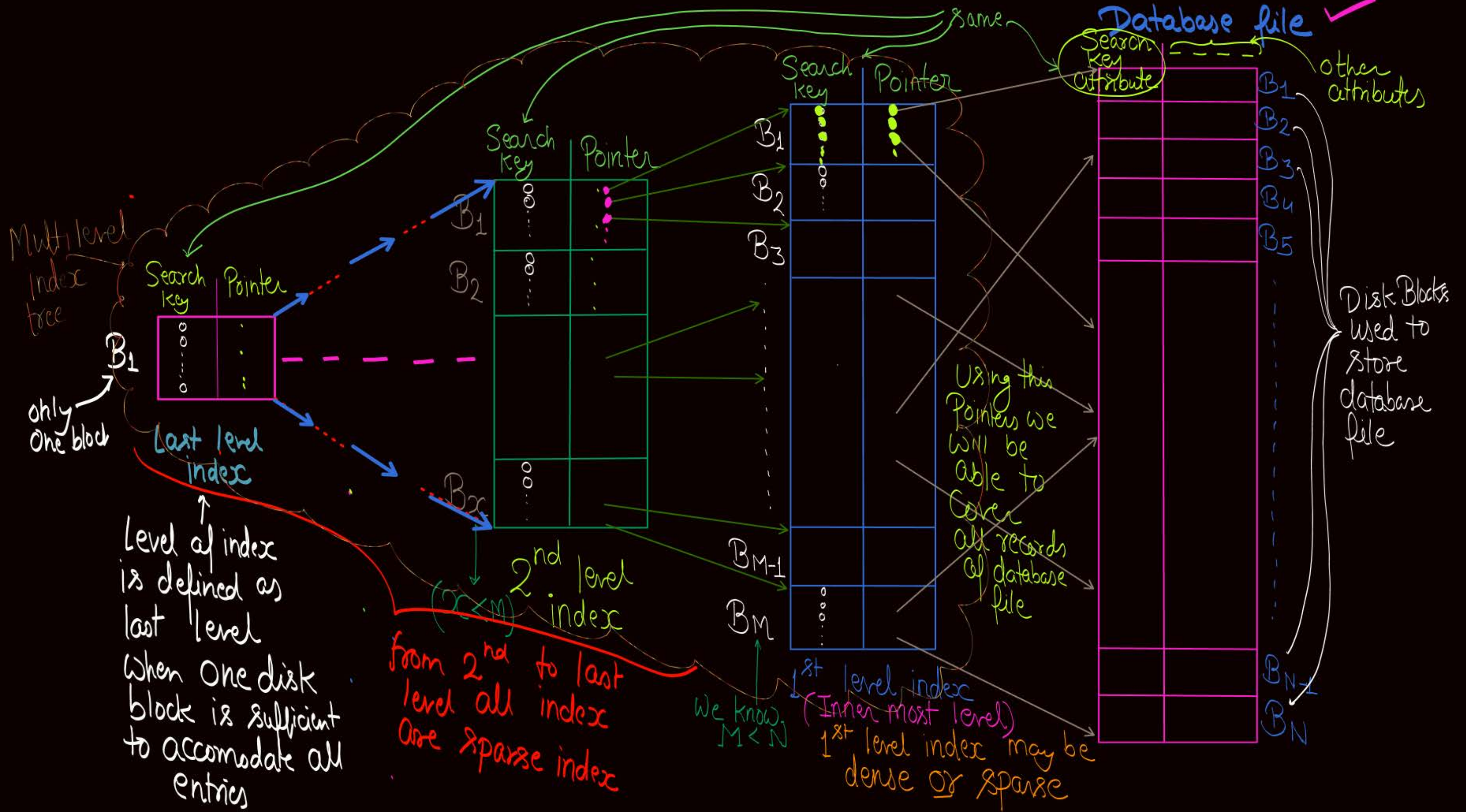
- Secondary index is always a dense index.

Note:- Any number of secondary index are possible on a database file { Because file don't need to be ordered }



Topic : Multi-level Index

- Multilevel index is used to reduce the IO Cost w.r.t single level index



Note:- Multi level index can reduce the IO cost significantly.

$$\text{IO Cost using multi-level Index} = \underbrace{\text{No. of levels in multi level index tree}} + \underbrace{1}$$

from each level of index tree, exactly one block will be access from secondary memory to main memory

1 Block of database file will be access from secondary memory to main memory in order to access the record corresponding to the search key



Topic : Multi-level Index

Using multi level index

$$\text{IO Cost} = \text{No. of levels} + 1$$

↳ for all access
{ Best case, worst case, avg case }
all same



Topic : Multi-level Index

In multi-level index,

- ① 1st level of index may be dense or may be sparse
- ② Except first level, all other levels of index are always sparse index

Note:- "Multi-level index" is suitable only for static database {i.e. the database in which insertion and/or deletion are rare}

But if database happens to be dynamic {i.e. insertion/deletion are frequent} then after every insertion or deletion we may have to restructure all the levels of multi-level index.
∴ Most of the time will be wasted in maintaining the multi-level index itself.

Note:- For dynamic database we use
dynamic multi-level search trees
i.e. B tree
B⁺ tree



Topic : Dynamic Multi-level Search Tree

→ B tree
→ B+ tree } These are
Balanced multi-way
Search tree

ie. all leaf nodes
are at same
level

it may be
m-ary search
tree where
'm' can be
any value



Topic : B Tree



Order of a node of B tree :- In general, Order of a node of B tree is defined as Maximum number of Child pointer a node of B tree can have

Consider a B tree of order = P { i.e.,

Structure of an internal node of B tree

- Maximum number of Child pointer a node can have = P
- Maximum number of Key values a node can have = $(P-1)$

All Keys within a node will always be in ascending order

Search Key value $\rightarrow$ Record pointer corresponding to search key value K_1

$$\{ \therefore K_1 < K_2 < K_3 \dots < K_{P-1} \}$$



Points to the Node of B tree at lower level that contains the information about Key value $< K_1$

$K_1 < \text{Key} < K_2$
Value

$K_2 < \text{Key} < K_3$
Value

$K_{P-2} < \text{Key} < K_{P-1}$
Value

$K_{P-1} < \text{Key}$
Values

Record Pointer :- Record pointers are used to point to the record in the database file corresponding to the associated key value.

Block Pointer :- Block pointers are used to point to a block(node) of B tree at lower level.
Also known as Child pointer / Node pointer / Tree pointer

Consider a B tree of order = P

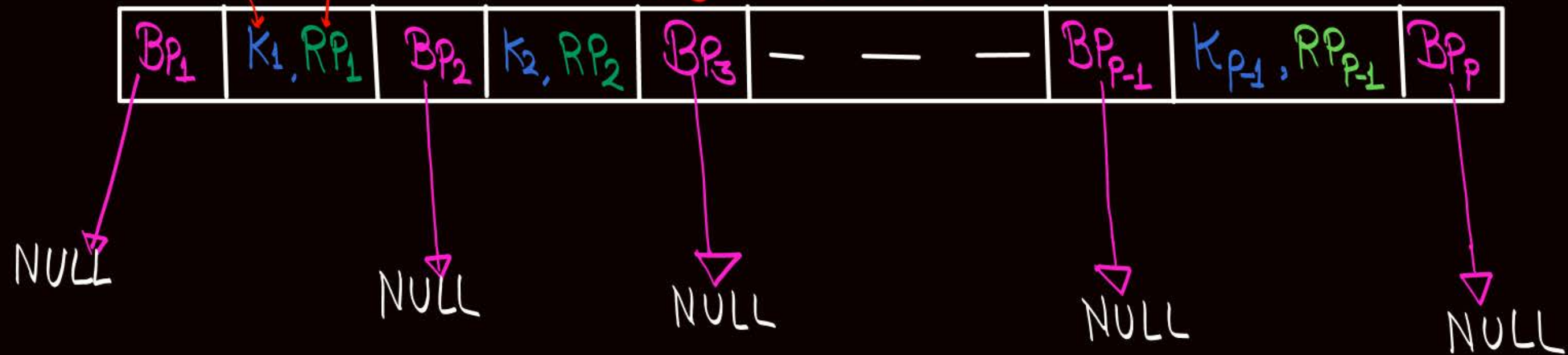
- { i.e.,
- Maximum number of Child pointer a node can have = P
 - Maximum number of Key values a node can have = $(P-1)$

Structure of a leaf node of B tree

- All keys within a node will always be in ascending order

Search Key value $\rightarrow$ Record pointer corresponding to search key value K_1

$\{ \therefore K_1 < K_2 < K_3 \dots < K_{P-1} \}$



Note:-

In B tree record pointers corresponding to the key values are present in leaf node of B tree, as well as in internal node of B tree

i.e., Structure of B tree is different from structure of normal multi-level index tree.

(In regular multi-level index tree all the record pointers are only at leaf level)

Note:-

In B tree each key value of database file will be present in exactly one node of B tree along with the record pointer corresponding to that key value, whereas in regular multi-level index tree a key value may be present in more than one index block

Note :-

Each node of B tree is a "disk block"
∴ Whatever information we want to store in a single node of B tree must fit within a single block of disk.

Consider a B tree of order = P } i.e.,

- Maximum number of Child pointer a node can have = P
- Maximum number of Key values a node can have = $(P-1)$

It is one node
of a B tree
of order = P



∴ it must fit
within the
block of disk

$$\Rightarrow \left\{ (P * \text{Block Pointer Size}) + (P-1) * (\text{Search Key Size} + \text{Record Pointer Size}) \right\} \leq \text{Disk Block Size}$$

↳ Using this equation we can obtain the maximum possible value of " P " (order)



Topic : B Tree



ie. 'K' is a search key

Consider a relations R with a key filed K. A B-tree of order P is used to access structures on K. Where 'P' denotes the maximum Number of tree pointers in a B-tree index node. Assume that key (k) is 30 bytes long disk block size 1024 byte. Each data pointer size P_D is 12 bytes long & each block pointer P_B is 10 byte long. In order for each B-Tree Node to fit in a single disk block, the maximum value of P is-

A 19

C X 21

B 20

D X 22

$$(P \times \text{Block Pointer} + (P-1) \times (\text{key size} + \text{Record Pointer size})) \leq \text{Block size}$$

$$P \times 10 + (P-1) \times (30 + 12) \leq 1024$$

$$52P \leq 1024 + 42$$

$$P \leq \frac{1066}{52} \Rightarrow P \leq 20.5 \dots$$

Hence $P \leq 20$ $\therefore$ Max Value = 20

Important

* Consider a B tree of order = 'P'

- ① Maximum number of Child Pointer a node of B tree Can have is = P
(Block Pointer)
- ② Maximum number of Key values/Record pointers a node of B tree Can have is = (P-1)
- ③ Minimum number of Child Pointer a root node of B tree must have is = 2
(Block Pointer)
- ④ Minimum number of Key values/Record pointers root node of B tree must have is = 1
- ⑤ Minimum number of Child Pointer a non-root node of B tree must have is = $\lceil \frac{P}{2} \rceil$
(Block Pointer)
- ⑥ Minimum number of Key values/Record pointers non-root node of B tree must have is = $(\lceil \frac{P}{2} \rceil - 1)$

it states that
index block must be
at least half filled {except root node}

HoWo

#Q.



a) Find the minimum number of nodes required in B tree of order = p and height = h

b) Find the minimum number of child pointers required in B tree of order = p and height = h

c) Find the minimum number of keys required in B tree of order = p and height = h

d) Find the maximum number of nodes possible in B tree of order = p and height = h

e) Find the maximum number of child pointers possible in B tree of order = p and height = h

f) Find the maximum number of keys possible in B tree of order = p and height = h

Consider
root node
is at
height = 0



Topic : B+ tree visualization

Structure of B+ tree is close to the structure of regular multi-level index tree.

Then Record with key value 3 will be in left sub-tree

Record with the value in an internal node is in right sub-tree

No record pointer is stored in internal node

Keys in internal node are dummy keys they are used only for searching the appropriate leaf node

if we say $key \leq 3$

$key < 3$

$3 \leq key < 5$

$5 \leq key$

leaf level

1	RP ₁	2	RP ₂	BP
---	-----------------	---	-----------------	----

3	RP ₃	4	RP ₄	BP
---	-----------------	---	-----------------	----

5	RP ₅	6	RP ₆	BP
---	-----------------	---	-----------------	----

NULL

Used to point the next block at leaf level

Record Pointer corresponding to key value = 1

All the keys of database file are present at leaf level along with their record pointers

Search key field

other attributes

2

1

3

5

6

4

Database file



Topic : B+ Tree



- * • B+ tree is some what similar to normal multi-level index.
- * • In B+ Tree all keys along with their record pointers are present at leaf nodes of B+ Tree.
- * *Keys within the node of B+ tree are store in ascending order*
- * • In B+ Tree keys in the internal nodes are used only for search operation (they are dummy keys), they are not used to point any record of the database as no record pointer is available in internal nodes of B+ Tree.



Topic : Order of Internal node of B+ Tree

Order of an internal node of B+ tree is defined as the maximum number of child pointer (block pointer/ node pointer) a B+ tree node can have. (same as B Tree)

Let order of a node of B+ tree is denoted by 'q'



Topic : Order of Internal node of B+ Tree

Let 'q' is the order of an internal node of B+ Tree

- Maximum number of child pointer a node can have is = q
- Maximum number of keys a node can have is = $q-1$
- Minimum number of child pointer a non-root node can have is = $\lceil q/2 \rceil$
- Minimum number of keys a non-root node can have is = $\lceil q/2 \rceil - 1$
- Minimum number of child pointer root node can have is = 2
- Minimum number of keys root node can have is = 1



Topic : Order of leaf node of B+ Tree

Order of leaf a node of B+ tree is defined as the maximum number of keys(record pointers) a B+ tree node can have. (different from B Tree)

Let order of a node of B+ tree is denoted by 'm'



Topic : Order of leaf node of B+ Tree

Let 'm' is the order of a leaf node of B+ Tree

- Maximum number of keys a leaf node can have is = m
- Minimum number of keys a non-root leaf node must have is = $\lceil m/2 \rceil$
- There will be exactly one block pointer in each leaf node, it will point to the next leaf node of B+ tree. (because of this pointer range query is more efficient in B+ tree)



Topic : NOTE



- If only one order is given in the question, then we will use the same order for both leaf node as well as for internal node.



2 mins Summary



Topic

Dynamic Multi level index

Topic

Structure of B tree

Topic

Structure of B+ tree

THANK - YOU